NESTED MUSIC TRANSFORMER: SEQUENTIALLY DECODING COMPOUND TOKENS IN SYMBOLIC MUSIC AND AUDIO GENERATION

Jiwoo Ryu[1] **Hao-Wen Dong**[2] **Jongmin Jung**[1] **Dasaem Jeong**[3]
[1]Dept. of Artificial Intelligence, [3]Dept. of Art & Technology, Sogang University, Seoul, South Korea
[2]University of California San Diego, US
{judejiwoo, jongmin, dasaemj}@sogang.ac.kr, hwdong@ucsd.edu

ABSTRACT

Representing symbolic music with compound tokens, where each token consists of several different sub-tokens representing a distinct musical feature or attribute, offers the advantage of reducing sequence length. While previous research has validated the efficacy of compound tokens in music sequence modeling, predicting all sub-tokens simultaneously can lead to suboptimal results as it may not fully capture the interdependencies between them. We introduce the Nested Music Transformer (NMT), an architecture tailored for decoding compound tokens autoregressively, similar to processing flattened tokens, but with low memory usage. The NMT consists of two transformers: the main decoder that models a sequence of compound tokens and the sub-decoder for modeling sub-tokens of each compound token. The experiment results showed that applying the NMT to compound tokens can enhance the performance in terms of better perplexity in processing various symbolic music datasets and discrete audio tokens from the MAESTRO dataset.

1. INTRODUCTION

The effectiveness of the autoregressive language model becomes dominant in generative tasks in various domains, including music. The language model has been the most widely used generative model in symbolic music generation [1–4]. After the success of vector quantization or residual vector quantization [5], the language model is also widely applied to audio-domain music generation [6–8].

The power of the language model comes from its autoregressive modeling of sequential information. Once the data is *flattened* to a sequence of discrete tokens, the language model can be applied in a straightforward manner. There have been many successive works on representing symbolic music data in a sequence of flattened tokens, such as MIDI-like encoding [9] or REMI [3].

However, a limitation of this approach is that the sequence length is quite lengthy, with the average number of tokens for pieces within the Lakh MIDI dataset [10] reaching 14,647. To overcome this limitation, the Compound Word Transformer [4] proposed an encoding scheme named Compound word that represents symbolic music as a sequence of compound tokens, in which several musical features or attributes are encoded into a single multi-dimensional token. By grouping musical features into two different compound token types, metric and note, Compound word shortens the sequence length to less than half of what is encoded with REMI as depicted in Figure 2. Similarly, Multitrack Music Transformer [11] employed a compound token scheme that encodes beat position, instrument, pitch, and duration into a single token, resulting in a sequence length approximately one-third of that encoded with REMI. Furthermore, note-level compound tokens demonstrated a clear advantage in performance for discriminative tasks such as identifying the genre or style of music and suggesting accompaniments [12].

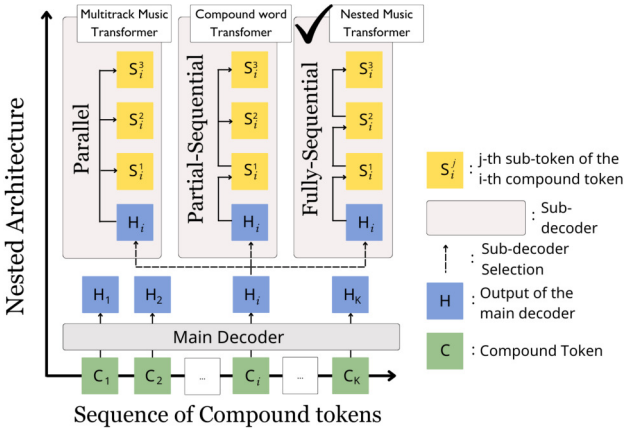


Figure 1: Diagram of the nested architecture with three different methods for predicting sub-tokens.

Despite these attempts to reduce the sequence length by packing musical features into a single compound token for various purposes, encoding schemes which flatten tokens like REMI are still dominant in symbolic music generation. Both [4] and [11] in symbolic music generation showed that the generation with REMI was favored in their listening tests. One of the causes is that the previous models are designed to predict multiple features in a parallel [11] or partial-sequential [4] way without considering interdependencies between different musical features encoded within

© J. Ryu, H-W. Dong, J. Jung, and D. Jeong. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** J. Ryu, H-W. Dong, J. Jung, and D. Jeong, "Nested Music Transformer: Sequentially Decoding Compound Tokens in Symbolic Music and Audio Generation", in *Proc. of the 25th Int. Society for Music Information Retrieval Conf.,* San Francisco, United States, 2024.

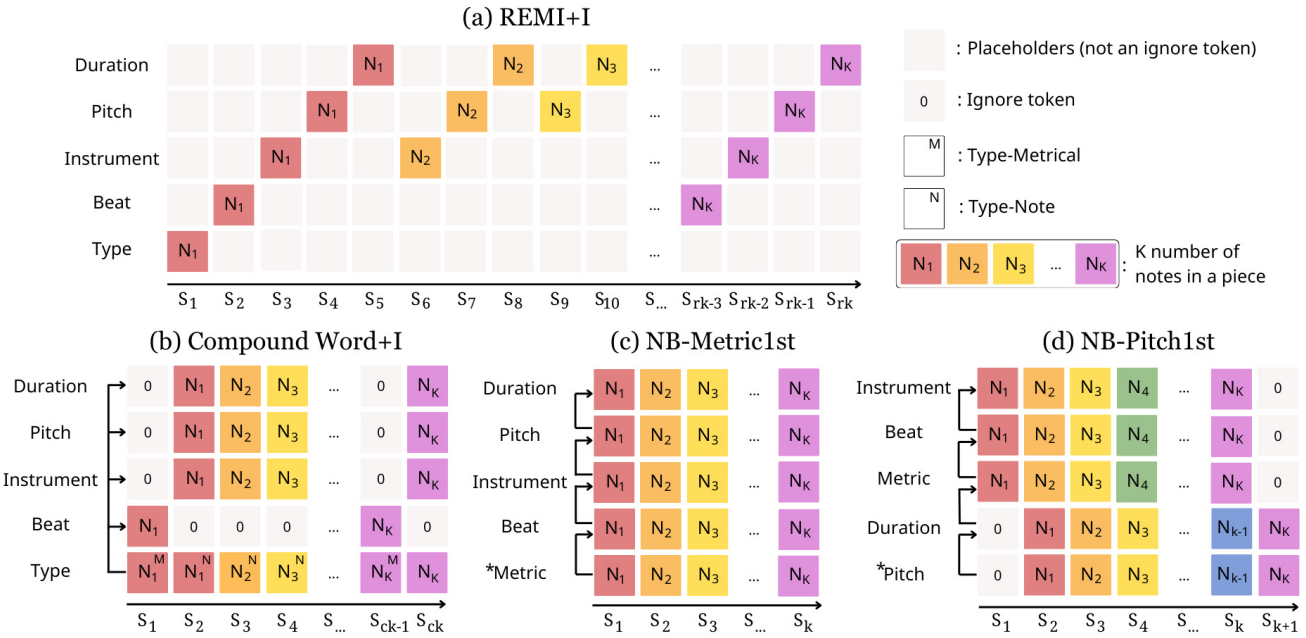


Figure 2: An example illustrating the proposed representations, note-based (NB) encoding (c) NB-Metric1st and (d) NB-Pitch1st, alongside REMI and Compound word. All encodings represent the same piece of music by using five musical features. Specifically, REMI and Compound word were not originally designed for multi-instrument pieces, which is why we renamed the encodings with "+I" to (a) and (b). Here, k denotes the number of notes and sequence length for NB, while r and c represent the ratios for REMI and Compound word, with values greater than 1.

the compound token, as depicted in Figure 1.

To address this challenge, we introduce a novel decoding framework called the Nested Music Transformer (NMT). The primary goal of this framework is to decode compound tokens in a fully sequential manner while maintaining efficient memory usage. The proposed NMT combines two distinct cross-attention architectures within its sub-decoder: the intra-token decoder and the Embedding Enricher. The intra-token decoder autoregressively decodes the sub-tokens of a single compound token, while the Embedding Enricher updates embedding of each sub-token by attending to the hidden states of previous compound tokens.

We demonstrated that our proposed architecture achieves performance comparable to that of flattening-based models, while requiring fewer computational resources in terms of GPU memory and training time. This was confirmed through both quantitative evaluations and subjective listening tests for symbolic music generation. Furthermore, our experiments showed that the NMT and other nested architectures perform similarly to strong baseline models when generating audio samples using discrete audio tokens. All source code, pretrained models and generated samples are available at `https://github.com/JudeJiwoo/nmt`.

2. NOTE-BASED ENCODING

Before we introduce the Nested Music Transformer, we explain Note-based encoding (NB), a compound token encoding scheme that we utilized as the primary encoding method. NB stands out for its ability to encapsulate the most comprehensive set of musical features within a single compound token, as illustrated in Figure 2.

2.1 Musical Features in Symbolic Encoding

As depicted in Figure 2, REMI, Compound word, and NB utilize several musical features to represent music pieces. We used a total of eight features: beat (position), pitch, and duration were essential, while instrument, chord, tempo, and velocity were selectively included based on the dataset characteristics. To encode other information, such as measure boundary and change in time signature, we also employed one additional feature *Type* or *Metric* following [4].

In Compound word (CP), musical features are categorized into two groups: "metrical" and "note." Consequently, the encoding employs two *Type* tokens to specify the group of each compound token. Unlike CP, NB does not require group indicator tokens however, since each note token in NB is assigned a beat, unlike REMI and CP, we designed the *Metric* feature to encode changes in the metrical structure. This allows the model to efficiently represent metrical changes within a single sub-token. Specifically, the *Metric* feature indicates whether the current note introduces a new time signature, measure, or beat, or continues the previous metrical context. For this purpose, we define four distinct values for the *Metric* feature vocabulary, each representing a different combination of metrical changes or continuations.

The *Beat* indicates the relative position of each note within a measure. The *Chord* was derived using a rule-based algorithm from [4]. The *Tempo* was set to follow an exponential scale for value changes, with this application varying across datasets. The *Instrument* feature specifies the instrument playing the note. In order to keep the variety of instruments manageable, we adopted the approach suggested in [11], trimming to 61 types of instruments. The

Pitch feature utilized 128 categories of pitch values represented in MIDI. The *Duration* refers to the length of time each note is played. The *Velocity* represents MIDI velocity (dynamics) of each note.

For the NB encoding method, a music piece P with K number of notes, $P = \{n_1, n_2, n_3, ..., n_K\}$ can be conceptualized as a sequence of compound tokens, denoted by $P_{nb} = \{x_1, x_2, x_3, \ldots, x_K\}$, wherein each event x_i is a compound token comprising up to eight sub-tokens in the orders like followings:

$$(x_i^{\text{metric}}, x_i^{\text{beat}}, x_i^{\text{chord}}, x_i^{\text{tempo}}, x_i^{\text{inst}}, x_i^{\text{pitch}}, x_i^{\text{dur}}, x_i^{\text{vel}})$$

2.2 Compound Shift

By reordering the sub-tokens within a compound token, we can position the target sub-token to be predicted first. This adjustment enhances the objective metric of the target sub-token, as it benefits from being processed primarily by the more powerful main decoder rather than the sub-decoder. Each event x_i which is shifted to pitch-first option comprises features like following:

$$(x_{i-1}^{\text{pitch}}, x_{i-1}^{\text{dur}}, x_{i-1}^{\text{vel}}, x_i^{\text{metric}}, x_i^{\text{beat}}, x_i^{\text{chord}}, x_i^{\text{tempo}}, x_i^{\text{inst}})$$

Note that the order of prediction of each sub-token in the entire *flattened* sequence does not change, and only the grouping boundary for a single compound token is shifted as depicted in Figure 2 (d). We will refer to the non-shifted representation as *NB-MF* and the shifted version as *NB-PF*.

3. NESTED MUSIC TRANSFORMER

In this section, we introduce the architecture of Nested Music Transformer (NMT), which is designed to handle compound tokens. The structure is composed of three primary components: token embedding, main decoder, and sub-decoder. The token embedding component summarizes the embeddings of each sub-token into a single vector which represents each compound token. Subsequently, the main decoder processes the sequence of these vectors using a decoder-only transformer architecture. Lastly, the sub-decoder decodes sub-tokens from the output of the main decoder. The proposed NMT integrates two distinct cross-attention architectures within its sub-decoder: the intra-token decoder and the Embedding Enricher. As the NMT generates sub-tokens, their embeddings are updated with contextual information by the Embedding Enricher, as illustrated in Figure 3.

3.1 Token Embedding & Main Decoder

To summarize multiple embeddings from each sub-token, we simply sum them along the sub-token axis following [6, 11]. Additionally, we integrate learnable absolute positional embedding [13] to denote the position of compound tokens within the sequence. Specifically, the i-th compound token x_i in the sequence is converted into a vector through the token embedding process and aggregated with its positional embedding. This combined vector is then fed into the main decoder, producing the output of the main decoder, also known as the hidden vector h_i.

3.2 Sub-decoder with Cross Attention

The main goal of the sub-decoder is to obtain proper hidden state to predict output sub-token s_i^j which is j-th sub-token of i-th compound token, based on output of the main decoder h_i and the preceding output sub-tokens $s_i^0, \ldots, s_i^{j-1}$ that are predicted before.

Many previous works have suggested using a similar sub-decoder to sequentially predict the sub-token sequence, such as updating hidden state by concatenating with the embedding of sub-tokens [4], using RNN [14] or causal self-attention [8]. However, through comparative experiments presented in Section 4, we found that applying cross-attention is one of the most effective way to model the compound token sequence in symbolic music.

The cross-attention-based sub-decoder operates by iteratively concatenating a key/value pair sequence K/V_i with embeddings of sub-tokens $\text{Emb}(s_i)$, starting with an initial key/value sequence that contains only the beginning-of-sequence BOS token. For each sub-token to be sampled, the architecture computes multi-head scaled dot-product cross-attention between the query sequence, consisting of positionally encoded output of the main decoder h_i, and the current key/value sequence. The positional encoding of h_i ensures that the hidden vector has a distinct bias for predicting target sub-token. From the attention output a_i^j, the matrix $\text{W}_{\text{logits}}^j$ is applied to create logits. This iterative process continues until all sub-tokens are sampled. The process can be expressed as follows:

$$\text{Query}_i^j = \text{PositionalEncoding}(h_i), \tag{1}$$

$$\text{K/V}_i^j = \begin{cases} BOS & \text{if } j = 0, \\ \text{Concat}(BOS, \ldots, \text{Emb}(s_i^{j-1})) & \text{if } j > 0, \end{cases} \tag{2}$$

$$a_i^j = \text{Cross-Attention}(\text{Query}_i^j, \text{K/V}_i^j), \tag{3}$$

$$s_i^j = \text{Sampling}(\text{Softmax}(a_i^j \text{W}_{\text{logits}}^j)) \tag{4}$$

3.2.1 Embedding Enricher

Since the embedding of a sub-token is a shallower vector compared to the output of the main decoder, we designed a cross-attention architecture called the Embedding Enricher. This architecture updates embedding of sub-token $\text{Emb}(s_i)$ with a context sequence derived from the prior outputs of the main decoder $h_{i-(w-1)}, ..., h_i$, where w represents the window size.

$$\text{Context}_i = \text{Concat}(BOS, h_{i-(w-1)}, ..., h_i), \tag{5}$$

$$\text{Enriched}_i = \text{Cross-Attention}(\text{Emb}(s_i), \text{Context}_i) \tag{6}$$

In the Nested Music Transformer, the output vector Enriched_i replaces the original embedding of sub-tokens before being concatenated into the key/value pair sequence in Equation (2) as depicted in Figure 3. These context-enriched embeddings allow the architecture to process attention with deeper vectors than the original embeddings, resulting in better performance on the objective metric compared to the standalone cross-attention-based sub-decoder, as demonstrated in Table 1.

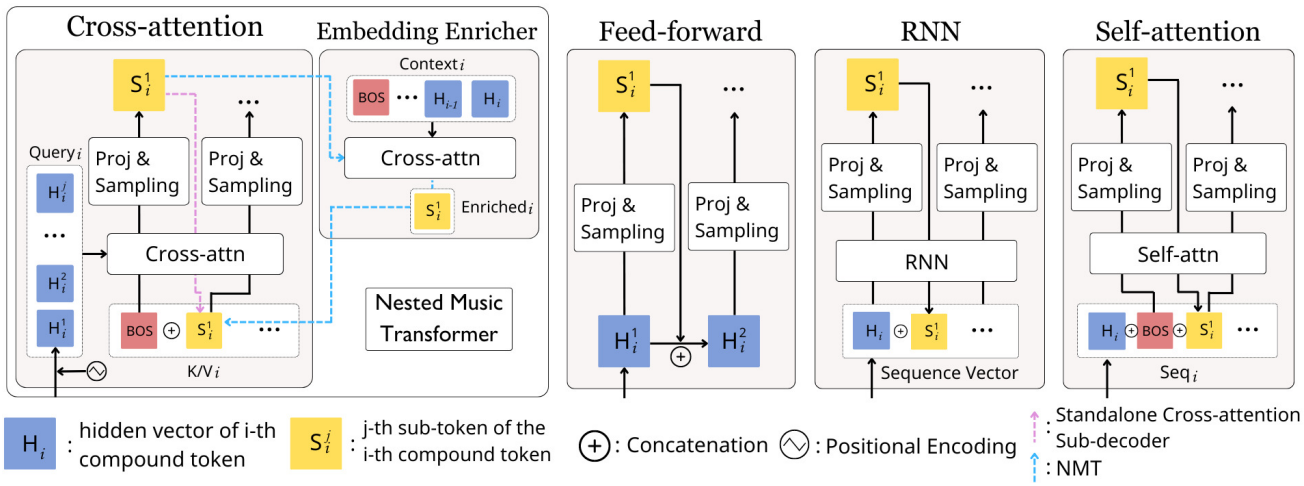


Figure 3: Illustrations of the proposed Nested Music Transformer (NMT) and other sub-decoder structures

3.3 Other Comparative Structures

3.3.1 Feed-forward-based Architecture

The Feed-forward-based sub-decoder, inspired by [4], iteratively updates the output of the main decoder to predict sub-tokens. It concatenates the previously used hidden state with the embedding of the last sampled output to predict the next sub-token.

3.3.2 RNN-based Architecture

RNN-based sub-decoder capitalizes on the sequential nature of recurrent neural network to update hidden state. The initial input sequence and hidden state utilize the output of the main decoder h_i, and through the iteration the embedding of the sampled output is appended to the input sequence until all the sub-tokens are generated.

3.3.3 Self-attention-based Architecture [1]

The self-attention-based sub-decoder aims to get the sequence vector Seq_i by iteratively concatenaing it with the embeddings of the sampled output $\text{Emb}(s_i)$. The initial sequence vector consists of the output of the main decoder h_i and BOS token to ensure that the initial attention values can be properly processed. This sequence vector Seq_i is then used as the query, key, and value in the self-attention mechanism. The process can be summarized as follows:

$$\text{Seq}_i^j = \begin{cases} \text{Concat}(h_i, BOS) & \text{if } j = 0, \\ \text{Concat}(h_i, BOS, ..., \text{Emb}(s_i^{j-1})) & \text{if } j > 0, \end{cases}, \tag{7}$$

$$a_i^j = \text{Self-Attention}(\text{Seq}_i^j), \tag{8}$$

$$s_i^j = \text{Sampling}(\text{Softmax}(a_i^j \text{W}_{\text{logits}}^j)) \tag{9}$$

[1] The proposed self-attention-based sub-decoder operates differently from the method described in [8]. Unlike ours, [8] used h_i as a base of every vector in the sequence, which is updated by the embedding of generated sub-tokens, similar to the operation of our proposed cross-attention-based sub-decoder. Experimental results indicate that the architecture in [8] outperforms our self-attention-based architecture and delivers comparable results to our cross-attention-based architecture.

3.4 Self-attention versus Cross-attention

The preference for cross-attention over self-attention arises from the observation that the output of the main decoder, h_i, already contains sufficient information to predict sub-tokens, as demonstrated in the parallel prediction method used in the Multitrack Music Transformer [11]. On the other hand, the embedding of the sampled output is comparatively shallow, lacking the previous context despite having the same dimension as h_i. Additionally, since both attention layers use a residual connection for the vectors used as keys, utilizing h_i as the key facilitates a direct gradient flow. Therefore, updating h_i as the key with cross-attention can be more advantageous than updating the embedding of the sampled sub-token with self-attention.

3.5 Applying to Audio Tokens

MusicGen [6] has employed a four-level residual vector quantization technique for a single token, which bears similarity to using four musical features or sub-tokens for compound tokens in symbolic music. Given that the optimal architecture, particularly for decoding compound features in a fully-sequential manner, is still being explored for audio tokens [8], we employed the Nested Music Transformer on discrete audio tokens to assess the potential of our proposed architecture.

4. EXPERIMENTS

4.1 Dataset Preparation

We selected four datasets to conduct our experiments on symbolic music generation: Pop1k7 [4], Pop909 [15], the Symbolic Orchestral Database (SOD) [16], and the clean version of the Lakh MIDI Dataset (LMD clean) [10], which is free of data leakage problems. During preprocessing, MIDI files without a time signature or with excessive or insufficient length were filtered out, and we specifically selected pieces featuring a minimum of four instruments for LMD clean. Note quantization varied across datasets: twelve resolutions per beat for SOD and four resolutions

	SOD						Lakh			Pop1k7			Pop909		
	GPU mem.(GB)	Time(s) / iter.	Token Len.	Mean↓	Beat	Pitch	Mean	Beat	Pitch	Mean	Beat	Pitch	Mean	Beat	Pitch
REMI [3]	19.90	0.461	6,638(±7,518)	**0.474**	**0.229**	**0.753**	**0.294**	0.293	**0.408**	**1.087**	**0.470**	**1.138**	**0.716**	0.368	0.984
CP [4]	7.93	0.119	3,230(±3,480)	0.604	0.257	0.971	0.361	0.288	0.527	1.172	0.495	1.219	0.911	0.410	1.220
CP* + NMT	16.13	0.224	–	0.545	0.237	0.864	0.327	0.288	0.466	1.103	0.483	1.154	0.724	0.334	**0.969**
NB-MF + Par. [11]	8.40	0.123	2,398(±2,764)	0.712	0.466	1.084	0.431	0.431	0.604	1.480	0.871	1.802	1.003	0.674	1.393
NB-MF + NMT	16.14	0.215	–	0.567	0.246	0.906	0.324	**0.276**	0.466	1.168	0.503	1.304	0.803	**0.264**	1.114
NB-PF + Par.	8.30	0.120	–	0.632	0.565	0.913	0.376	0.502	0.481	1.396	0.998	1.604	0.986	0.824	1.359
NB-PF + CA	14.74	0.174	–	0.564	0.276	0.867	0.305	0.287	0.424	1.161	0.538	1.244	0.767	0.357	1.052
NB-PF + NMT	16.13	0.217	–	0.549	0.263	0.855	0.306	0.285	0.427	1.149	0.515	1.243	0.771	0.345	1.090
NB-PF + FF	8.12	0.122	–	0.607	0.361	0.881	0.338	0.372	0.449	1.280	0.635	1.396	0.850	0.431	1.121
NB-PF + RNN	9.77	0.144	–	0.591	0.300	0.915	0.315	0.297	0.437	1.166	0.531	1.257	0.792	0.366	1.077
NB-PF + SA	15.67	0.181	–	0.574	0.287	0.902	0.311	0.287	0.431	1.204	0.553	1.320	0.849	0.417	1.150

CP*: Compound word representation **NB-MF**: metric-first NB **NB-PF**: pitch-first NB **NMT**: cross-attention-based sub-decoder + Embedding Enricher **CA**: cross-attention-based sub-decoder **FF**: Feed-forward-based sub-decoder **SA**: self-attention-based sub-decoder

Table 1: Model comparison on their average NLL loss for symbolic music. The GPU memory usage and iteration times for each model in SOD is included. Additionally, we included the average token length and standard deviation across all pieces in SOD.

per beat for the others. We also filtered out MIDI files with expressive tempo and timing. We split the prepared data, reserving 10% for validation and 10% for testing. Additionally, augmentation techniques for pitch and chord involved random semitone shifts $s \in \mathbb{Z}$ within a range of $s \sim U(-5, 6)$.

4.2 EnCodec for MAESTRO

For discrete audio tokens, we prepared MAESTRO dataset [9], which has 200 hours piano performance audio files. We fine-tuned the audio tokenizer proposed by [6] with MAESTRO audio files to create sequences of discrete audio tokens, each with 30 seconds of length. The sampling rate of the token is 50 Hz, which means 30 seconds of audio is represented with 1500 audio tokens, each with 4 different codebooks.

4.3 Model and Hyperparameter Configuration

The baseline models for symbolic music generation are defined as follows: flattening for REMI [3], partial-sequential Feed-forward-based sub-decoder for Compound word [4], and parallel prediction with NB-MF [11]. Additionally, the *delay* method proposed by [6] is explored as a baseline for generating audio tokens, which utilizes rearranged residual vectors or sub-tokens in a parallel manner. In exploring both symbolic music and audio token generation, we conducted experiments using the Nested Music Transformer (NMT) and various sub-decoder architectures to assess the effectiveness of our proposed model. To ensure a fair comparison among these models, we aimed for a comparable number of model parameters, approximately 40 million for symbolic music and 62 million for discrete audio tokens [2]. To enhance efficiency in processing long sequences within the transformer architecture, we integrated Flash attention [17].

Training the model entailed 100K steps for symbolic music and 200k for discrete audio tokens, utilizing the AdamW optimizer [18], with a segment batch size of 8 and 16 for each task, where β_1 were set to 0.9, β_2 to 0.95, and a gradient clipping threshold was set to 1.0. We implemented a cosine learning rate schedule with a warm-up phase of 2000 or 4000 steps for each task. During this warm-up phase, the learning rate gradually increased before reaching its maximum value $1 \times \mathrm{e}^{-4}$. To address overfitting concerns, we applied dataset-specific dropout rates instead of using early stopping. These dropout rates were chosen to ensure that the optimal validation loss remained stable until the end of training. We utilized mixed precision techniques.

[2] Both models have 8 attention heads and a dimension size of 512, with a single layer for all sub-decoder architectures and an additional single layer for the Embedding Enricher when using the NMT. However, they have a total of 12 and 15 decoder layers, respectively.

4.4 Quantitative Evaluation on Symbolic Music

We evaluated the symbolic music generation task using the average negative log-likelihood (NLL). However, directly comparing the loss values across models using different encoding schemes posed challenges. To address this, we first adjusted the input sequence length for each encoding scheme to ensure that the NLL is derived from a similar amount of context regardless of the encoding scheme. Furthermore, instead of calculating the average NLL as done during the training steps, we calculated it based on the set of probabilities of tokens processed with full context. To achieve this, we used a moving-window method with a window size equal to the input sequence length to create a set of overlapping input sequences.

Secondly, we adjusted the probabilities for each subtoken in a compound token to account for discrepancies between REMI and other encoding schemes like CP and NB. REMI omits redundant tokens such as repetitive positions (beat), as depicted in Figure 2. Thus, when predicting a new note, a model based on REMI must decide whether to add the note at a new position by predicting a new beat token, or to add the note at the same position by predicting a pitch token. In contrast, CP and NB, due to the nature of their encoding schemes, split this prediction into two steps: first, they determine the beat position, and then they predict the pitch. This means they have more prior information when predicting the pitch token since changes in beat are already fixed and provided as a condition. To ad-

	FAD-uncon↓	FAD-cond↓	KLD↓	mean NLL↓
Parallel	0.166	0.206	0.075	4.669
Flatten	0.140	**0.176**	0.068	4.482
Delay [6]	0.168	0.188	0.066	4.564
Self-attention	**0.131**	0.186	0.074	4.353
Cross-attention	0.145	0.190	**0.065**	**4.314**
NMT	0.165	0.198	0.067	4.318

Table 2: Model comparison for discrete audio tokens

	Coherence↑	Richness↑	Consistency↑	Overall↑
	Mean(±margin of error)			
REMI [3]	3.18 ± 0.20	3.33 ± 0.18	3.33 ± 0.18	3.17 ± 0.18
CP [4]	2.94 ± 0.22	3.24 ± 0.18	2.97 ± 0.20	3.06 ± 0.20
CP + NMT	3.22 ± 0.19	3.35 ± 0.17	$\mathbf{3.39} \pm 0.17$	3.32 ± 0.17
NB-PF + NMT	$\mathbf{3.37} \pm 0.19$	$\mathbf{3.44} \pm 0.18$	3.37 ± 0.19	$\mathbf{3.36} \pm 0.20$

Table 3: Results of subjective listening test, presenting mean values with 95% confidence intervals.

just the probability of sub-tokens in NB and CP, which differ due to the discrepancy, we accumulated the probability of each sub-token to the next token in NB or CP if that sub-token was omitted in its corresponding REMI encoding. For example, when predicting a pitch token at the same beat, $P(\text{pitch} \mid \text{context})$ in REMI can be compared to $P(\text{same_beat} \mid \text{context}) \times P(\text{pitch} \mid \text{context}, \text{same_beat})$ in NB or CP.

From the results, we observe several key tendencies. First, applying the Nested Music Transformer (NMT) enhances the overall performance across all types of compound token encodings, including previously suggested schemes like CP and NB-MF (similar to [11]). Second, the NMT demonstrates a clear advantage in using the cross-attention-based sub-decoder and the Embedding Enricher compared to other baseline architectures. Finally, our pitch-first NB (NB-PF) encoding outperforms the metric-first NB (NB-MF) encoding in predicting pitch. This is because the model can predict the next pitch feature through the main decoder by leveraging the previously inferred note position information. Conversely, NB-MF showed lower loss in beat prediction. This difference arises from which sub-token relationships are calculated through the main decoder instead of the sub-decoder. Overall, the results indicate that pitch-first token grouping is an efficient strategy.

4.5 Quantitative Evaluation on Discrete Audio Tokens

We evaluated models with discrete audio tokens using following metrics: Fréchet Audio Distance (FAD), Kullback-Leibler Divergence (KL), and the mean NLL loss over sequences. A lower FAD score suggests that the generated audio is more plausible. To mitigate sample number bias for the test set, we employed adaptive FAD as proposed by [19], along with CLAP [20] embeddings for each sample. FAD scores were computed based on 500 unconditionally generated samples and 345 samples generated given prompts. Following [6], we computed the KL-divergence over the probabilities of the labels between the original and the generated audio samples. Table 2 shows the evaluation results.

We observe that using a cross-attention-based sub-decoder or the NMT achieves better NLL compared to a self-attention-based sub-decoder. However, the tendency differs from that seen in symbolic music. Adding the Embedding Enricher did not significantly improve performance in the audio domain. We hypothesize that this disparity arises from the distinct characteristics of tokens in both domains. In the symbolic domain, each musical feature requires context to form sufficient semantic information, whereas each token in the audio domain, with a 2048 vocabulary size codebook, contains more standalone information. This observation suggests potential avenues for future research, such as exploring effective methods to integrate the semantic information of symbolic music with discrete audio tokens.

4.6 Subjective Listening Test

For the subjective listening test, we used the Symbolic Orchestral Database (SOD) [16] to generate MIDI samples given four-measure prompts. We carefully selected eight prompts from the test split and generated continuation results using four different models: two baseline models (REMI and CP) and two proposed models (CP + NMT and NB-PF + NMT). We applied different sampling methods to each model. [3] We conducted the test with 29 participants, asking them to evaluate the generated outputs based on three criteria: *Coherence* (the naturalness of transitions), *Richness* (the variety of harmony and rhythm), and *Consistency* (the lack of errors in composition), as well as an *Overall* rating for the perceptual quality of the samples as a whole.

As summarized in Table 3, our proposed models generated samples of comparable quality to REMI, outperforming the baseline CP. The smaller gap between REMI and NB + NMT in the subjective listening test compared to the teacher-forcing NLL evaluation suggests that NB + NMT may be more robust to exposure bias during sequence generation. Another possible explanation is that compound tokens are more effective at capturing the given context, as also demonstrated in the experiments of [4].

5. CONCLUSION

In summary, this work presents the Nested Music Transformer, an advanced architecture that decodes compound tokens in music generation, applicable to both in the symbolic and audio domain. Our architecture distinguishes itself by addressing the twin challenges of sequence length and feature interdependencies through a nested transformer setup that efficiently manages GPU resources and training processes. The experiments validate the competitiveness of our model over previous methods, achieving on par results in both objective metrics and subjective listening tests while lowering training costs.

[3] During the generation process, we used nucleus sampling (top-p sampling) with $p = 0.99$. Our proposed models were sensitive to the choice of the temperature parameter, where an improperly selected temperature would result in excessive repetition regardless of encoding schemes. Therefore, we searched for the optimal temperature value for each model within the range of [1.0, 1.3] on the validation set.

6. ACKNOWLEDGEMENTS

This research was supported by the National R&D Program through the National Research Foundation of Korea (NRF) funded by the Korean Government (MSIT) (RS-2023-00252944, Korean Traditional Gagok Generation Using Deep Learning).

7. REFERENCES

[1] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, I. Simon, C. Hawthorne, N. Shazeer, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck, "Music transformer: Generating music with long-term structure," in *International Conference on Learning Representations*, 2018.

[2] Y. Ren, J. He, X. Tan, T. Qin, Z. Zhao, and T.-Y. Liu, "Popmag: Pop music accompaniment generation," in *Proceedings of the 28th ACM international conference on multimedia*, 2020, pp. 1198–1206.

[3] Y.-S. Huang and Y.-H. Yang, "Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions," in *Proceedings of the 28th ACM international conference on multimedia*, 2020, pp. 1180–1188.

[4] W.-Y. Hsiao, J.-Y. Liu, Y.-C. Yeh, and Y.-H. Yang, "Compound word transformer: Learning to compose full-song music over dynamic directed hypergraphs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 1, 2021, pp. 178–186.

[5] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *Transactions on Machine Learning Research*, 2023.

[6] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Defossez, "Simple and controllable music generation," in *Advances in Neural Information Processing Systems*, A. Oh, T. Neumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 47 704–47 720.

[7] G. Le Lan, V. Nagaraja, E. Chang, D. Kant, Z. Ni, Y. Shi, F. Iandola, and V. Chandra, "Stack-and-delay: a new codebook pattern for music generation," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 796–800.

[8] D. Yang, J. Tian, X. Tan, R. Huang, S. Liu, H. Guo, X. Chang, J. Shi, J. Bian, Z. Zhao *et al.*, "Uniaudio: Towards universal audio generation with large language models," in *Forty-first International Conference on Machine Learning*, 2024.

[9] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the maestro dataset," *arXiv preprint arXiv:1810.12247*, 2018.

[10] C. Raffel, *Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching*. Columbia University, 2016.

[11] H.-W. Dong, K. Chen, S. Dubnov, J. McAuley, and T. Berg-Kirkpatrick, "Multitrack music transformer," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[12] M. Zeng, X. Tan, R. Wang, Z. Ju, T. Qin, and T. Liu, "Musicbert: Symbolic music understanding with large-scale pre-training," in *Findings of the Association for Computational Linguistics: ACL/IJCNLP 2021, Online Event, August 1-6, 2021*, ser. Findings of ACL, vol. ACL/IJCNLP 2021. Association for Computational Linguistics, 2021, pp. 791–800.

[13] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," in *International conference on machine learning*. PMLR, 2017, pp. 1243–1252.

[14] Z. Wang, Y. Zhang, Y. Zhang, J. Jiang, R. Yang, G. Xia, and J. Zhao, "PIANOTREE VAE: structured representation learning for polyphonic music," in *Proceedings of the 21th International Society for Music Information Retrieval Conference, ISMIR 2020, Montreal, Canada, October 11-16, 2020*, J. Cumming, J. H. Lee, B. McFee, M. Schedl, J. Devaney, C. McKay, E. Zangerle, and T. de Reuse, Eds., 2020, pp. 368–375. [Online]. Available: http://archives.ismir.net/ismir2020/paper/000096.pdf

[15] Z. Wang, K. Chen, J. Jiang, Y. Zhang, M. Xu, S. Dai, and G. Xia, "POP909: A pop-song dataset for music arrangement generation," in *Proceedings of the 21th International Society for Music Information Retrieval Conference, ISMIR 2020, Montreal, Canada, October 11-16, 2020*, 2020, pp. 38–45.

[16] L. Crestel, P. Esling, L. Heng, and S. McAdams, "A database linking piano and orchestral MIDI scores with application to automatic projective orchestration," in *Proceedings of the 18th International Society for Music Information Retrieval Conference, ISMIR 2017, Suzhou, China, October 23-27, 2017*, 2017, pp. 592–598.

[17] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.

[18] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.

[19] A. Gui, H. Gamper, S. Braun, and D. Emmanouilidou, "Adapting frechet audio distance for generative music evaluation," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 1331–1335.

[20] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.